

Software Design Description (SDD) for SENA Network

Document Version: 1.0.0

1. Introduction

1.1 Purpose

This Software Design Description (SDD) provides a detailed technical design for the SENA Network, a Layer 2 (L2) modular infrastructure. It elaborates on the architectural components, module designs, data structures, and algorithms necessary to implement the requirements outlined in the Software Requirements Specification (SRS). This document is intended for developers, architects, and technical stakeholders involved in the construction and maintenance of the SENA Network.

1.2 Scope

This SDD covers the design of the core SENA L2 protocol, including its sequencer, state management, and integration with Aptos L1. It details the design of the Keyless Authentication Engine, Social Connect Identity Index, Multi-Asset Gas Paymaster Engine, and Optimistic L2 Governance Layer, with a particular focus on the Rust implementation using Sovereign SDK and RocksDB, and the integration of the “Aptos-Level Secure” model.

1.3 Definitions, Acronyms, and Abbreviations

Refer to Section 1.3 of the Software Requirements Specification (SRS) for a comprehensive list of definitions, acronyms, and abbreviations.

1.4 References

- [1] Software Requirements Specification (SRS) for SENA Network
- [2] SENA Network: Layer 2 Modular Infrastructure Documentation (Stakeholder Document)
- [3] Sovereign SDK Documentation. *Sovereign Labs*. Available at: <https://docs.sovereign.xyz/>
- [4] RocksDB. *GitHub*. Available at: <https://github.com/facebook/rocksdb>
- [5] Aptos Keyless Accounts. *Aptos Docs*. Available at: <https://aptos.dev/concepts/accounts/#keyless-accounts>
- [6] NOMT: Nearly-Optimal Merkle Trie. *GitHub*. Available at: <https://github.com/aptos-labs/aptos-core/tree/main/storage/apptosdb/src/schema/nomt>

2. System Architecture

2.1 High-Level Architecture

(A Mermaid diagram illustrating the system architecture was intended here but removed due to rendering issues. A textual description follows.)

SENA Network operates as a Sovereign SDK-based rollup, with its core logic implemented in Rust. It functions as a single-sequencer L2, periodically committing its state to the Aptos L1 for security and finality. The architecture is designed for modularity, performance, and resource efficiency.

2.2 Component Breakdown

2.2.1 SENA L2 Sequencer

- **Role:** The central processing unit of the L2. It receives transactions from client applications, orders them, executes them against the Sovereign SDK runtime, and periodically commits state roots to Aptos L1.
- **Technology:** Rust application, leveraging Sovereign SDK components.
- **Key Responsibilities:**
 - Transaction reception and mempool management.
 - Transaction ordering and block production.
 - Interaction with the Sovereign SDK runtime for state transitions.
 - Generation and submission of State Roots to Aptos L1.
 - Integration with the Aptos Signature Verification Module for ZKP validation.

2.2.2 Sovereign SDK Runtime

- **Role:** Provides the execution environment for SENA's custom state transition logic. It defines the modules (Keyless Auth, Social Connect, Gas Paymaster, Governance) and their interactions.
- **Technology:** Rust modules, built using the Sovereign SDK framework.
- **Key Responsibilities:**
 - Managing the L2 state, including accounts, balances, and module-specific data.
 - Executing transaction logic according to the defined state transition function.
 - Interfacing with RocksDB for persistent storage.
 - Providing APIs for module interactions.

2.2.3 RocksDB + NOMT Storage Layer

- **Role:** The persistent storage solution for the L2 state. RocksDB provides a high-performance key-value store, while NOMT ensures efficient Merkle tree operations for state proofs.
- **Technology:** RocksDB (C++ library with Rust bindings), NOMT (Rust implementation).
- **Key Responsibilities:**
 - Storing all L2 state data (accounts, balances, module data).

- Maintaining a Merklized representation of the state for efficient proof generation.
- Providing fast read and write access for the Sovereign SDK runtime.

2.2.4 Aptos Signature Verification Module

- **Role:** A dedicated module within the L2 sequencer responsible for verifying Zero-Knowledge Proofs (ZKPs) for keyless authentication.
- **Technology:** Rust implementation, importing Aptos's open-source signature verification code.
- **Key Responsibilities:**
 - Receiving pre-computed ZK proofs from client applications.
 - Performing lightweight cryptographic verification of ZK proofs.
 - Validating the link between Web2 JWTs and ephemeral public keys.

2.2.5 Aptos L1 Move Contract

- **Role:** A smart contract deployed on the Aptos L1 blockchain that serves as the security anchor for SENA. It receives and immutably records the L2 State Roots.
- **Technology:** Aptos Move language.
- **Key Responsibilities:**
 - Receiving and storing L2 State Roots.
 - Providing a mechanism for users to withdraw funds directly from L1 using L2 state proofs.
 - Managing the L1 bridge for asset transfers between L1 and L2.

2.2.6 Aptos Decentralized Prover Network

- **Role:** An external, decentralized network responsible for generating computationally intensive Zero-Knowledge Proofs (ZKPs) required for keyless authentication.
- **Technology:** External service, provided by Aptos.
- **Key Responsibilities:**
 - Receiving requests for ZKP generation from client applications.
 - Computing ZK circuits based on Web2 JWTs and ephemeral public keys.
 - Returning pre-computed ZK proofs to client applications.

3. Module Design

3.1 Protocol-Enforced Keyless Authentication Engine

- **Design Principles:** Security, privacy, user-friendliness, and off-chain computation.
- **Components:**
 - **Client-Side Key Pair Generator:** Generates ephemeral key pairs upon user login via Web2 providers.

- **ZKP Request Handler:** Client-side logic to request ZK proofs from the Aptos Decentralized Prover Network.
- **Aptos Signature Verification Module (L2 Node):** As described in Section 2.2.4, this module performs the on-chain verification of ZK proofs.
- **Account Derivation Logic:** Implements the deterministic L2 address derivation using `$aud` , `$sub` , and a pepper value.
- **Flow:**
 1. User logs in via Google/Apple on a client application.
 2. Client generates an ephemeral key pair.
 3. Client requests a ZK proof from the Aptos Decentralized Prover Network, providing the JWT and EPK.
 4. Prover Network computes and returns the ZK proof to the client.
 5. Client sends a transaction to the SENA L2 sequencer, including the EPK and ZK proof.
 6. SENA L2 sequencer's Aptos Signature Verification Module verifies the ZK proof.
 7. If valid, the transaction is processed, and the L2 address is derived and associated with the user's session.

3.2 Social Connect Identity Index

- **Design Principles:** Privacy-preserving, fast lookup, multi-mapping support.
- **Data Structure:** A key-value store within RocksDB, where keys are cryptographic hashes of human-readable identifiers, and values are SENA L2 addresses. A separate index might map SENA L2 addresses to multiple hashed social identifiers.
- **Hashing Algorithm:** SHA-256 with a network-wide salt to prevent rainbow table attacks.
- **API:** RPC endpoint for `resolve_address(hashed_identifier)` and `get_social_links(sena_address)` .

3.3 Multi-Asset Gas Paymaster Engine

- **Design Principles:** Flexibility, user convenience, economic stability.
- **Components:**
 - **Whitelisted Stablecoin Ledger:** A state variable within the Sovereign SDK runtime that stores the list of approved stablecoins and their current exchange rates to APT.
 - **Gas Calculation Logic:** Determines the gas cost of a transaction in terms of APT, then converts it to the equivalent amount in the chosen stablecoin.
 - **Fee Extraction Logic:** Deducts the stablecoin fee directly from the user's transaction payload or account balance.
 - **System-Controlled Vault:** A dedicated L2 account that collects stablecoin fees.
 - **Aptos L1 Bridge Integration:** Logic within the sequencer to interact with the Aptos L1 Move contract to convert collected stablecoins to APT and fund L1 state commitments.
- **Flow:**

1. User submits a transaction, specifying a preferred whitelisted stablecoin for gas.
2. Sequencer calculates gas cost in APT.
3. Sequencer converts APT cost to stablecoin equivalent using current exchange rates.
4. Sequencer extracts stablecoin fee from user.
5. Stablecoin fees are deposited into the System-Controlled Vault.
6. Periodically, the sequencer initiates a cross-chain transaction via the Aptos L1 bridge to convert a portion of the vault's stablecoins to APT to cover L1 state commitment costs.

3.4 Optimistic L2 Governance Layer

- **Design Principles:** Decentralization, efficiency, low overhead.
- **Components:**
 - **Staking Module:** Manages `$SENA` token staking for voting participation.
 - **Off-Chain Voting Platform:** (External to L2 node) Where token holders cast votes.
 - **Governance Multi-Signature Council:** A pre-defined set of trusted entities responsible for signing valid off-chain vote results.
 - **Administrative Module (L2 Node):** Ingests signed payloads, verifies signatures, and applies state updates.
- **Flow:**
 1. Token holders stake `$SENA` and vote on parameter changes on an off-chain platform.
 2. If a vote passes, the result (e.g., new stablecoin whitelisted) is formatted into a payload.
 3. The Governance Multi-Signature Council signs the payload.
 4. The signed payload is submitted to the SENA L2 sequencer.
 5. The Administrative Module verifies the council's signatures.
 6. Upon successful verification, the system variables in the Sovereign SDK runtime are updated.

4. Data Design

4.1 Data Structures

- **Account State:**

```
struct Account {  
    address: L2Address,  
    balance: HashMap<AssetId, u128>,  
    // ... other account-related data  
}
```

- **Social Link Mapping:**

```
struct SocialLink {  
    hashed_identifier: HashedIdentifier,  
    l2_address: L2Address,  
    // ... metadata like provider, timestamp  
}
```

- **Whitelisted Assets:**

```
struct WhitelistedAsset {  
    asset_id: AssetId,  
    symbol: String,  
    exchange_rate_to_apt: u128, // Fixed-point representation  
}
```

- **Governance Parameters:**

```
struct GovernanceParameter {  
    key: String,  
    value: String, // Or specific type based on parameter  
    last_updated_block: u64,  
}
```

4.2 Database Schema (RocksDB)

RocksDB will be used as a key-value store. Column families will be utilized to logically separate different types of data, improving query performance and organization.

Column Family Name	Key Type	Value Type	Description
accounts	L2Address	Account (serialized)	Stores individual user account states.
social_links_by_hash	HashedIdentifier	L2Address	Maps hashed social IDs to L2 addresses.
social_links_by_address	L2Address	Vec<HashedIdentifier>	Maps L2 addresses to multiple hashed social IDs.
whitelisted_assets	AssetId	WhitelistedAsset	Stores details of approved stablecoins.
governance_params	String (param name)	GovernanceParameter	Stores network configuration parameters.
state_roots	BlockNumber	StateRootHash	Records L2 state roots committed to Aptos L1.
transactions	TxHash	Transaction	Stores historical L2 transactions.

4.3 Merkle Tree Implementation (NOMT)

NOMT will be integrated with RocksDB to provide a merklized state. This allows for efficient generation of state proofs, crucial for L1 commitments and user fund withdrawals. The NOMT will maintain a Merkle tree over the entire L2 state, enabling quick verification of state changes without recomputing the entire tree.

5. Security Design

5.1 Inherited Security from Aptos L1

- **State Root Commitment:** The L2 sequencer will periodically compute a cryptographic hash of its entire state (the State Root) and submit it to a designated Move contract on the Aptos L1. This process leverages Aptos's robust consensus mechanism to ensure the immutability and finality of the L2 state history.
- **Emergency Withdrawal Mechanism:** The L1 Move contract will include functionality allowing users to initiate withdrawals of their L2 funds directly from L1, using a valid L2 state proof and their Aptos L1 address. This mechanism acts as a safety net, guaranteeing fund recoverability even if the L2 sequencer is compromised or offline.

5.2 Keyless JWT Validation Architecture

- **Ephemeral Key Pairs:** Client applications will generate short-lived ephemeral key pairs for each user session, enhancing security by limiting the exposure window of any single key.
- **Decentralized ZKP Generation:** The computationally intensive task of generating ZK proofs will be offloaded to the Aptos Decentralized Prover Network. This prevents the L2 sequencer from becoming a bottleneck or a target for denial-of-service attacks related to ZKP computation.
- **On-Chain ZKP Verification (Rust):** The SENA L2 node will incorporate Aptos's battle-tested, open-source Rust libraries for ZKP signature verification. This verification is lightweight and performed efficiently during transaction processing, ensuring the authenticity of keyless transactions without significant performance overhead.

5.3 Privacy-Preserving Identifiers

- **Hashed Social Identifiers:** All human-readable social identifiers will be stored as SHA-256 hashes, combined with a unique network salt. This prevents direct exposure of personal information in the public L2 state while still allowing for unique identification and mapping.
- **OIDC-based Address Derivation:** L2 addresses will be deterministically derived from OIDC claims (`$aud` , `$sub`) and a pepper value, ensuring that user identities are linked to their blockchain addresses in a secure and privacy-preserving manner.

6. Interface Design

6.1 External Interfaces

- **RPC API:** Standard JSON-RPC interface for client applications to submit transactions, query state, and interact with L2 modules.
- **Aptos L1 Bridge Interface:** A Rust module within the sequencer that interacts with the Aptos L1 Move contract for state root commitments and asset transfers.
- **Client-Side SDKs:** Libraries (e.g., JavaScript, Mobile) for developers to easily integrate with SENA's keyless authentication and other features.

6.2 Internal Interfaces

- **Sovereign SDK Module Interfaces:** Well-defined Rust traits and structs for inter-module communication within the Sovereign SDK runtime.
- **RocksDB Interface:** Rust bindings for interacting with the RocksDB key-value store.

7. Future Considerations

- **Decentralized Sequencer:** While initially a single-sequencer model, future iterations may explore decentralized sequencer designs for enhanced censorship resistance and fault tolerance.
 - **Fraud Proofs:** Implementation of fraud proofs for a more robust optimistic rollup model.
 - **Additional L1 Integrations:** Potential integration with other L1 blockchains beyond Aptos.
-

References

[1] Software Requirements Specification (SRS) for SENA Network. *Manus AI*. [2] SENA Network: Layer 2 Modular Infrastructure Documentation. *Manus AI*. [3] Sovereign SDK Documentation. *Sovereign Labs*. Available at: <https://docs.sovereign.xyz/> [4] RocksDB. *GitHub*. Available at: <https://github.com/facebook/rocksdb> [5] Aptos Keyless Accounts. *Aptos Docs*. Available at: <https://aptos.dev/concepts/accounts/#keyless-accounts> [6] NOMT: Nearly-Optimal Merkle Trie. *GitHub*. Available at: <https://github.com/aptos-labs/aptos-core/tree/main/storage/aptosdb/src/schema/nomt> [7] AptosDB: The Aptos Database. *Aptos Docs*. Available at: <https://aptos.dev/nodes/aptos-node-references/aptosdb/> [8] Merkle Trees. *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Merkle_tree [9] Zero-Knowledge Proofs. *Wikipedia*. Available at: https://en.wikipedia.org/wiki/Zero-knowledge_proof [10] OIDC and ZKP in Aptos. *Aptos Labs Blog*. Available at: <https://aptoslabs.com/blog/aptos-keyless-accounts-oidc-zkp>